FLOWDIARY

AI-Powered Ethical Hacking

Lecture 6: How the Internet Works

Clients & servers · DNS · IP addresses · TCP/IP · ports · the full journey from URL to rendered page

Tutor: Engr. Ibrahim Auwal

> **Why this lecture matters for ethical hackers**
>
> Almost every attack and defense you will study touches the network. If you do not understand how a request travels from a browser to a server and back, terms like "DNS spoofing," "port scanning," "man-in-the-middle," or "TLS interception" stay abstract.
>
> This lecture builds the mental map. Once you can trace a single click end to end, you can reason about where an attacker could listen, inject, redirect, or impersonate — and where a defender can detect and block.

Learning Objectives

By the end of this lecture you should be able to:

- Explain the client–server model and identify which is which in a real interaction.
- Describe what an IP address is, the difference between IPv4 and IPv6, and public vs private addresses.
- Explain how DNS turns a human-readable name into an IP address, step by step.
- Describe the role of TCP/IP, the idea of layers, and what a "packet" is.
- Explain what ports are and why they matter for both services and attackers.
- Trace the complete journey of typing a URL until a page renders, naming each component involved.

1. The Client–Server Model

The internet is, at its core, a conversation between two kinds of programs: clients and servers. Almost everything else is detail layered on top of this simple idea.

1.1 What is a client?

A client is any program that initiates a request because it wants something. Your web browser is a client. When you open a page, the browser is saying, in effect, "Please send me this resource." Other examples of clients include a mobile app fetching your messages, an email program checking your inbox, or a command-line tool like curl.

The key idea is that the client starts the conversation. It knows what it wants and goes looking for it.

1.2 What is a server?

A server is a program that waits for requests and responds to them. It does not start conversations; it listens. When a request arrives, the server figures out what is being asked for, does any necessary work — looking up a database, reading a file, running code — and sends a response back.

The word "server" is used loosely. Sometimes it means the software (a "web server" like Nginx or Apache), and sometimes it means the physical or virtual machine the software runs on. In practice both meanings coexist; context tells you which one someone means.

> **Analogy: the restaurant**
>
> **Think of a restaurant.** You (the client) decide you want food and place an order. The kitchen (the server) is always there, waiting for orders. It does not cook until someone asks. When your order arrives, the kitchen prepares it and sends a plate back. You never see the kitchen's internal work — you only see the result.
>
> **Security lens:** an attacker might pose as a fake kitchen (a malicious server), intercept the order on its way (a man-in-the-middle), or send the kitchen a poisoned order designed to break it (an injection attack).

1.3 Requests and responses

Every interaction is a pair: a request goes out, a response comes back. On the web this is governed by HTTP (HyperText Transfer Protocol). A request says what you want and includes metadata; the response includes the content plus a status code that tells the client how things went.

HTTP status	Meaning	What it tells you
200 OK	Success	The request worked and content follows.
301 / 302	Redirect	The resource lives elsewhere; go there instead.
403 Forbidden	Access denied	The server understood but refuses — interesting during recon.
404 Not Found	Missing	No such resource at this path.
500 Server Error	Server broke	Something failed server-side — sometimes leaks clues.

2. IP Addresses — How Machines Are Located

For a client to talk to a server, it needs to know where the server is. On the internet, "where" is an IP address — a numeric label that uniquely identifies a device on a network. Think of it as the postal address of a machine.

2.1 IPv4

The older and still most common format is IPv4. It looks like four numbers separated by dots, for example 142.250.190.78. Each number ranges from 0 to 255, which means an IPv4 address is 32 bits long. That gives roughly 4.3 billion possible addresses.

That sounded like a lot in the 1980s. It is not enough for today's world of phones, laptops, servers, and smart devices, which is why we ran out and invented workarounds (and IPv6).

2.2 IPv6

IPv6 is the newer format, designed to solve the shortage. It is 128 bits long and written as eight groups of hexadecimal digits, for example 2001:0db8:85a3:0000:0000:8a2e:0370:7334. The address space is so vast it is effectively unlimited for any practical purpose.

2.3 Public vs private addresses

Not every address is reachable from the open internet. Certain ranges are reserved as private — they are reused inside homes and offices and are not routable on the public internet. Your laptop at home almost certainly has a private address.

Type	Example ranges	Where you see it
Private	10.x.x.x, 172.16–31.x.x, 192.168.x.x	Inside home / office networks.
Public	Everything else routable	Assigned by your ISP, visible to the internet.
Loopback	127.0.0.1 ("localhost")	Refers to your own machine.

> **NAT: how many devices share one public address**
>
> Your home router uses Network Address Translation (NAT). All your devices have private addresses behind the router, but to the outside world they appear to share the router's single public address. The router keeps a table mapping internal conversations to external ones so replies find their way back.
>
> **Why hackers care:** NAT hides internal structure. Scanning a public address does not directly reveal the devices behind it — which is why attackers try to get a foothold inside the network first.

3. DNS — The Internet's Address Book

Humans remember names like google.com, not numbers like 142.250.190.78. DNS (the Domain Name System) is the system that translates names into IP addresses. It is often called the phone book or address book of the internet.

3.1 Why DNS exists

Names are stable and meaningful; IP addresses can change and are hard to memorize. DNS lets a company move its servers to new IPs without anyone having to relearn the name. You type the name, DNS quietly resolves it to whatever IP is correct right now.

3.2 The DNS lookup, step by step

When you type a name, your computer asks a chain of servers until it finds the answer. Here is the journey:

1. **Browser / OS cache.** First your machine checks if it already knows the answer from a recent lookup. If so, it stops here.
2. **Recursive resolver.** If not cached, the query goes to a resolver (usually run by your ISP or a service like 8.8.8.8). This resolver does the legwork on your behalf.
3. **Root servers.** The resolver asks a root server, which does not know the final answer but knows who handles ".com" addresses.
4. **TLD servers.** The resolver then asks the ".com" top-level-domain server, which knows which name server is responsible for google.com.
5. **Authoritative name server.** Finally the resolver asks google.com's own authoritative server, which returns the actual IP address.
6. **Answer returned and cached.** The IP travels back to your machine, which caches it so the next lookup is instant.

> **Security lens: DNS is a prime attack surface**
>
> **DNS spoofing / cache poisoning:** if an attacker feeds a resolver a fake answer, your browser is sent to a malicious server while the address bar still shows the real name.
>
> **DNS as recon:** attackers enumerate subdomains (mail., vpn., dev.) to map an organization's footprint.
>
> **DNS tunneling:** data can be smuggled in and out of a network hidden inside DNS queries, bypassing weak firewalls.

3.3 Common DNS record types

Record	Purpose
A	Maps a name to an IPv4 address.
AAAA	Maps a name to an IPv6 address.
CNAME	Alias: points one name at another name.

Record	Purpose
MX	Mail server for the domain.
TXT	Free text; used for verification, SPF, etc.
NS	Which name servers are authoritative for the domain.

4. TCP/IP — The Rules of Conversation

Knowing the server's address is not enough. Both sides need a shared set of rules for how to package, send, and reassemble data. That set of rules is the TCP/IP protocol suite — the foundation the entire internet runs on.

4.1 The idea of layers

Networking is organized into layers, where each layer has one job and relies on the layer below it. This separation means an application does not need to know how electrical signals work, and a cable does not need to understand web pages. A simplified four-layer view:

Layer	Job	Examples
Application	Speaks the language apps use	HTTP, HTTPS, DNS, SMTP
Transport	Reliable (or fast) delivery, ports	TCP, UDP
Internet	Addressing and routing across networks	IP, ICMP
Link	Moving bits on the local physical network	Ethernet, Wi-Fi

4.2 What is a packet?

Data is not sent as one giant blob. It is broken into small chunks called packets. Each packet carries a piece of the data plus headers — labels that say where it came from, where it is going, and how to put it back together. Packets may take different routes across the internet and arrive out of order; the receiving side reassembles them.

4.3 TCP vs UDP

Two main transport protocols sit on top of IP, and they make opposite trade-offs.

	TCP	UDP
Reliability	Guarantees delivery & order	No guarantee; packets may drop
Speed	Slower (more overhead)	Faster (less overhead)
Connection	Handshake first, then data	Just sends, no handshake
Used for	Web, email, file transfer	Streaming, gaming, DNS, voice

The TCP three-way handshake

Before TCP sends real data, both sides agree to talk using three messages:

1. SYN — client says "I want to talk, here is my starting number."

2. SYN-ACK — server replies "Okay, I acknowledge you, here is my number."

3. ACK — client confirms "Acknowledged — let's go."

Security lens: a SYN flood attack sends many SYNs but never completes the handshake, exhausting the server's connection table. Port scanners also abuse the handshake to learn which ports respond.

5. Ports — Many Doors on One Address

A single machine runs many services at once: a web server, an email server, a database. They all share one IP address, so how does incoming traffic know which service to reach? The answer is ports.

5.1 What a port is

A port is a number from 0 to 65535 that identifies a specific service on a machine. If the IP address is the street address of a building, the port is the apartment number inside it. A full destination is therefore an IP address plus a port, written like 142.250.190.78:443.

5.2 Well-known ports

Some port numbers are standardized so clients know where to knock by default.

Port	Service	Notes
80	HTTP	Unencrypted web traffic.
443	HTTPS	Encrypted web traffic (TLS).
22	SSH	Secure remote login — a major attacker target.
53	DNS	Name resolution.
25 / 587	SMTP	Sending email.
3389	RDP	Windows remote desktop — frequently attacked.

Why ports are central to hacking

Port scanning (e.g. with Nmap) is usually the first active step in reconnaissance. By probing which ports are open, an attacker learns which services are running and therefore which vulnerabilities might apply.

An open port is a potential door. Every listening service is something that can be fingerprinted, tested for weak configuration, or exploited. Defenders close unused ports to shrink this attack surface.

6. The Full Journey: From URL to Rendered Page

Now we put everything together. Here is what actually happens, in order, when you type https://www.example.com into your browser and press Enter. Each step uses a concept from the earlier sections.

7. **Parse the URL.** The browser breaks the URL into parts: the scheme (https), the hostname (www.example.com), and the path. The scheme tells it to use encryption and default to port 443.
8. **DNS resolution.** The browser needs an IP address for the hostname. It checks its cache, then asks a resolver, which walks the DNS chain (root → TLD → authoritative) until it returns the IP. (Section 3.)
9. **Open a TCP connection.** Using the IP and port 443, the browser performs the TCP three-way handshake with the server. (Section 4.)
10. **TLS handshake.** Because this is HTTPS, the two sides negotiate encryption: they verify the server's certificate and agree on keys, so the rest of the conversation is private and tamper-resistant.
11. **Send the HTTP request.** Over the encrypted channel, the browser sends a request like GET / for the page, along with headers describing itself. (Section 1.)
12. **Server processes and responds.** The web server receives the request, possibly queries a database or runs code, and returns an HTTP response with a status code and the HTML content.
13. **Browser renders the page.** The browser parses the HTML, then discovers it needs more resources — CSS, JavaScript, images — and repeats the whole cycle (often steps 2–6) for each one until the page is complete.

> **One click, every concept**
> Notice that a single page load touched everything in this lecture: the client–server model (1), IP addresses (2), DNS (3), TCP/IP and the handshake (4), and ports (5). This is why understanding the journey is so powerful — it ties the whole picture together.
>
> **For the ethical hacker:** every numbered step above is a place where an attacker could interfere — poisoning DNS, intercepting the handshake, stripping TLS, injecting into the request, or exploiting the server. Knowing the sequence tells you where to look.

7. Recap and Key Takeaways

- **Clients ask, servers answer.** The client always initiates; the server listens and responds.
- **IP addresses locate machines;** IPv4 is running out, IPv6 is the long-term answer, and private addresses hide behind NAT.
- **DNS turns names into IPs** by walking a chain of servers — and it is a major attack surface.

- **TCP/IP is the rulebook;** data travels as packets, and TCP guarantees delivery via a handshake while UDP trades reliability for speed.
- **Ports are the doors** on a machine; scanning them is step one of reconnaissance.
- **The URL-to-page journey** strings all of these together, and every step is a potential point of attack or defense.

Discussion & Lab Prompts

1. Run nslookup or dig on a domain you use daily. Which records come back? Try it twice and watch caching speed it up.
2. Use your browser's developer tools (Network tab) to load a simple page and count how many separate requests it actually makes.
3. Find your own private IP address and your public IP. Why are they different? What sits between them?
4. Discuss: at which single step in the URL-to-page journey would you place a defense to catch the most attacks, and why?

Key Terms Glossary

Term	Quick definition
Client	Program that initiates a request.
Server	Program that listens and responds.
IP address	Numeric label locating a device on a network.
DNS	System that translates names into IP addresses.
Resolver	Server that performs DNS lookups on your behalf.
TCP/IP	The core protocol suite of the internet.
Packet	A small labelled chunk of transmitted data.
Handshake	The TCP setup exchange (SYN, SYN-ACK, ACK).
Port	Number identifying a specific service on a machine.
NAT	Lets many private devices share one public IP.